

BCSE309L

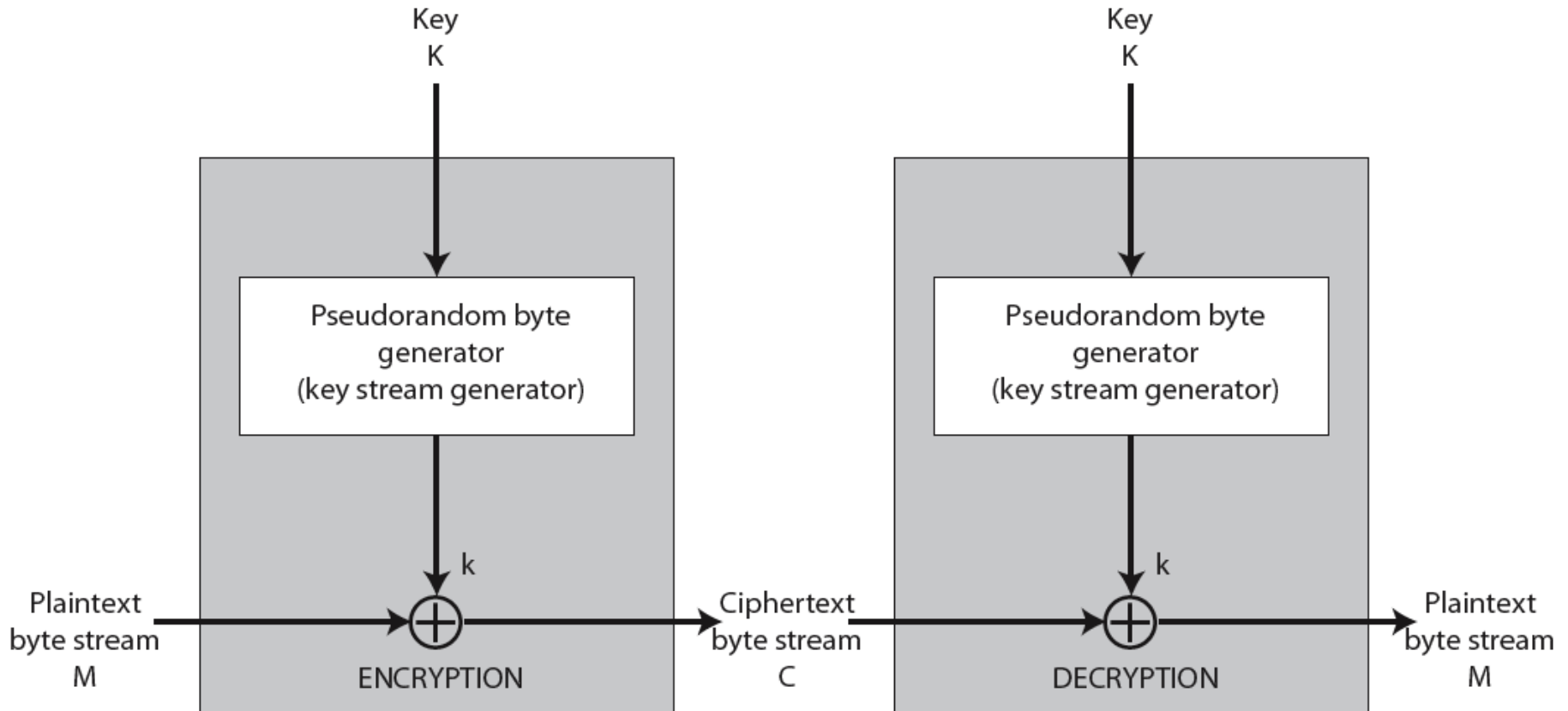


Module 2- Pseudorandom Number Generation and Stream Ciphers

Stream Ciphers

- process message bit by bit (as a stream)
- have a pseudo random **keystream**
- combined (XOR) with plaintext bit by bit
- Idea of randomness of **stream key** completely destroys statistically properties in message
 - $C_i = M_i \text{ XOR } \text{StreamKey}_i$
- but must never reuse stream key
 - otherwise can recover messages

Stream Cipher Structure



Stream Cipher Properties

- some design considerations are:
 - long period with no repetitions
 - statistically random
 - depends on large enough key
 - large linear complexity
- properly designed, can be as secure as a block cipher with the same size key
- but usually simpler & faster

RC4

- RC4 is a stream cipher designed in 1987 by Ron Rivest.
- a proprietary cipher owned by RSA, kept secret, released at the sites of Cyberpunk remailers.
- simple but effective, variable key size from 1 to 256 bytes
- byte-oriented stream cipher
- Used in
 - SSL/TLS (Secure socket, transport layer security) between web browsers and servers,
 - IEEE 802.11 wireless LAN std: WEP (Wired Equivalent Privacy), WPA (WiFi Protocol Access) protocol

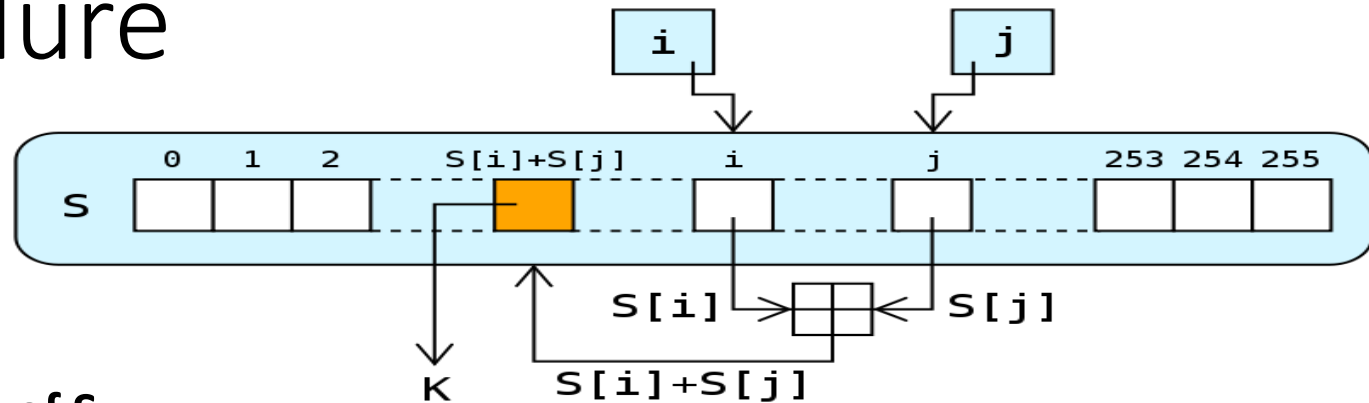
RC4

- key forms random permutation of all 8-bit values
- uses that permutation to scramble input info processed a byte at a time
- Runs faster - five/fifteen times than DES/3DES
- the period of the cipher is overwhelmingly likely to be greater than 10^{100}

RC4 Procedure

- A variable-length key of from 1 to 256 bytes (8 to 2048 bits) is used to initialize a 256-byte state vector S , with elements $S[0], S[1], \dots, S[255]$.
- For encryption and decryption, a byte K is generated from S by selecting one of the 255 entries in a systematic fashion.
- As each a new generated value of K , the entries in S are once again permuted.

RC4 Procedure



Step 1: Initialization of S

(a) the entries of S are set equal to the values from 0 through 255 in scending order; that is; $S[0] = 0$, $S[1] = 1, \dots, S[255] = 255$.

(b) A temporary vector, T , is also created. If the length of the key K is 256 bytes, then K is transferred to T . Otherwise, for a key of length **keylen** bytes, the first **keylen** elements of T are copied from K and then K is repeated as many times as necessary to fill out T .

```
for i = 0 to 255 do
    S[i] = i
    T[i] = K[i mod keylen]) // extend key
```

RC4 Procedure

- **Step 2: Initial Permutation of S**
- Next we use **T** to produce the initial permutation of **S**. This involves starting with **S[0]** and going through to **S[255]**, and, for each **S[i]**, swapping **S[i]** with another byte in **S** according to a scheme dictated by **T[i]**:

```
j = 0
for i = 0 to 255 do           //update swap
    j = (j + S[i] + T[i]) (mod 256)
    swap (S[i], S[j])
```

- Because the only operation on **S** is a swap, the only effect is a permutation. **S** still contains all the numbers from 0 through 255.

RC4 Procedure

- **Step3: Stream Generation**

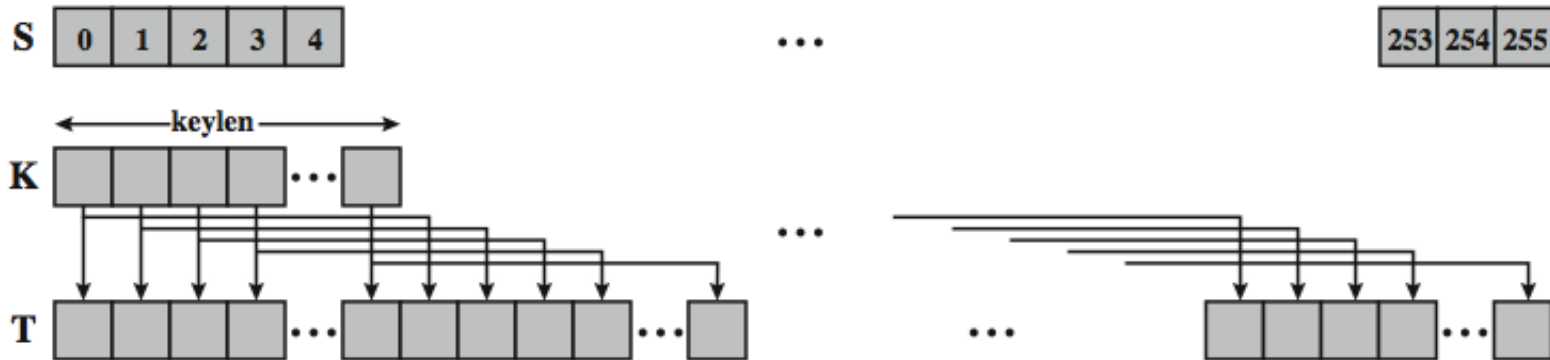
- Once the **S** vector is initialized, the input key is no longer used. Stream generation involves cycling through all the elements of **S**[i], and, for each **S**[i], swapping **S**[i] with another byte in **S** according to a scheme dictated by the current configuration of **S**. After **S**[255] is reached, the process continues, starting over again at **S**[0]:

```
i = j = 0
for each message byte Mi
    i = (i + 1) (mod 256)    //rotate thru S
    j = (j + S[i]) (mod 256) //rotate swap
    swap(S[i], S[j]) //update permutation S
    t = (S[i] + S[j]) (mod 256) //pick key index
    Ci = Mi XOR S[t]      //encrypt
```

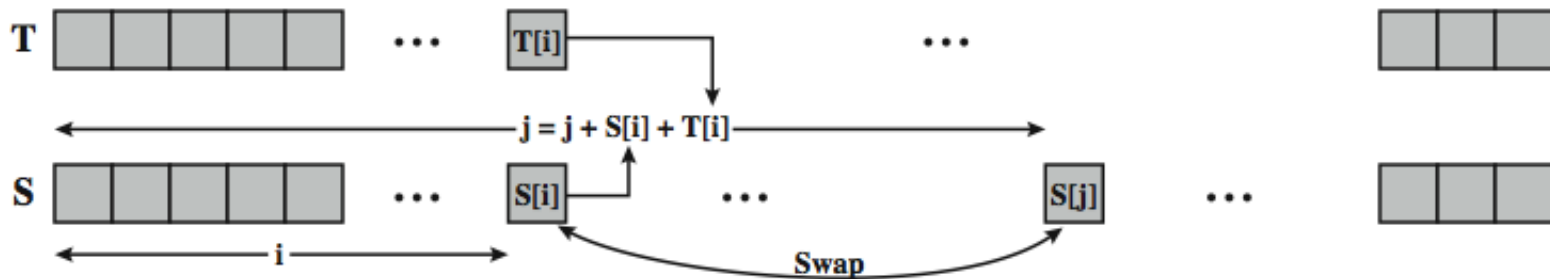
- **Step 4: Encryption**

- To encrypt, XOR the value **K** with the next byte of plaintext.
To decrypt, XOR the value **K** with the next byte of ciphertext.

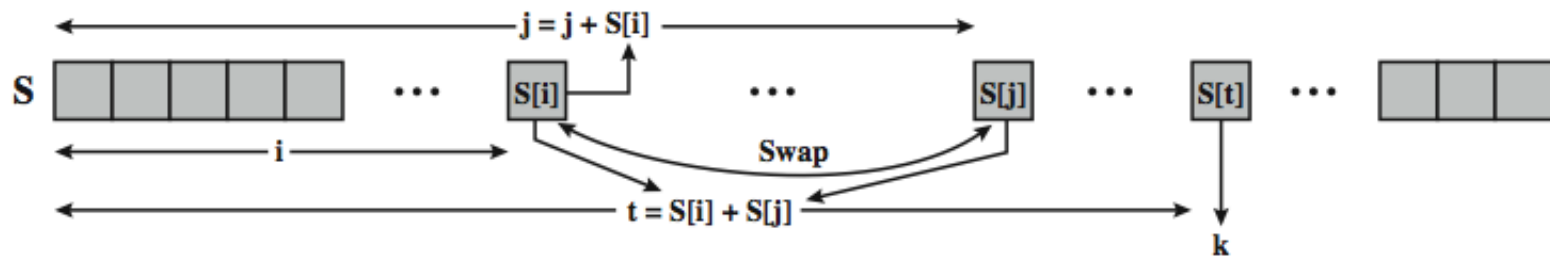
RC4 Overview



(a) Initial state of S and T



(b) Initial permutation of S

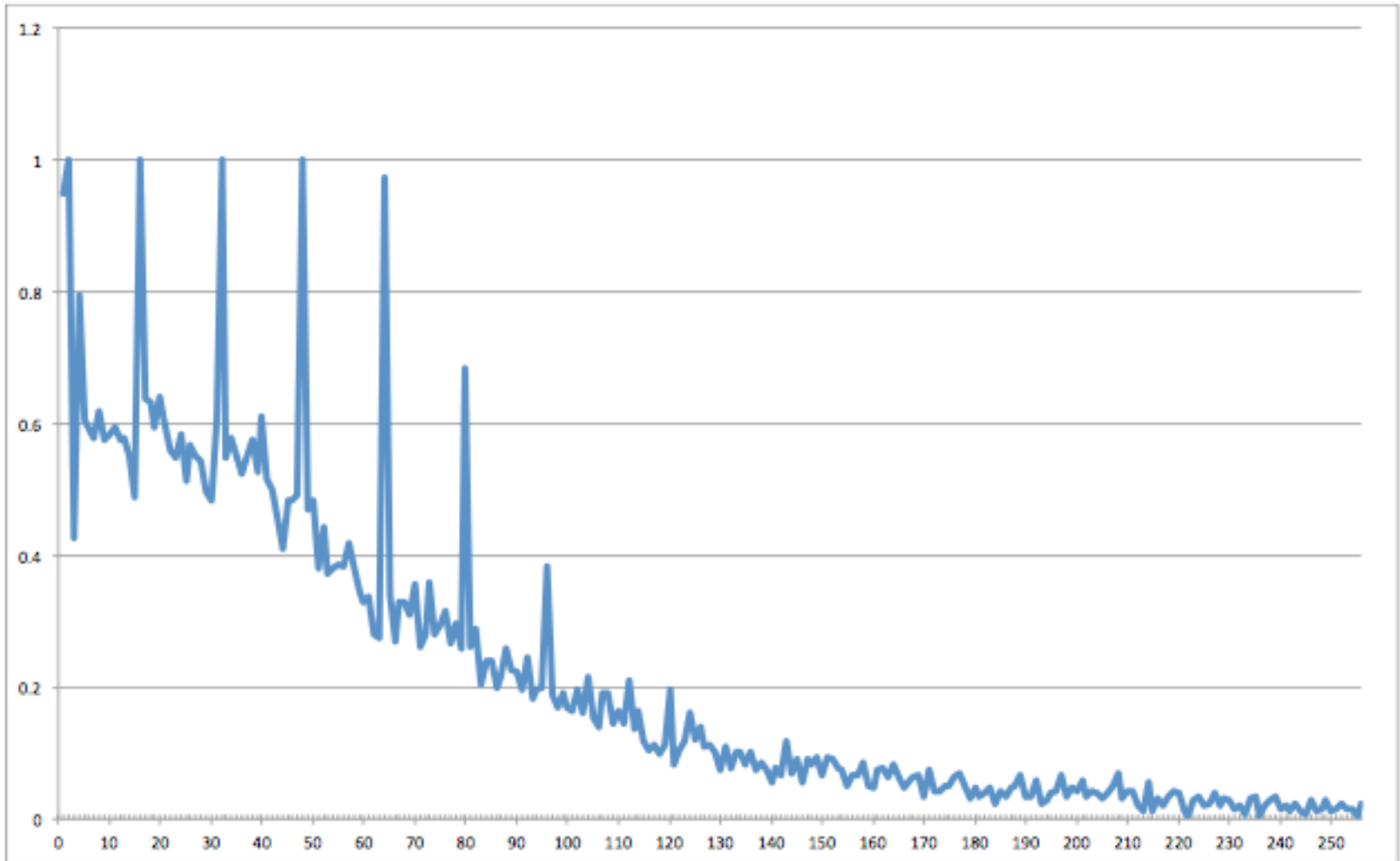


(c) Stream Generation

RC4 Security

- claimed secure against known attacks
 - However, have some analyses, some on the verge of practical
 - first 256 bytes have bias
 - multiple keys/same plaintext attack
- result is very non-linear
- since RC4 is a stream cipher, must **never reuse a key**
- have a concern with WEP, but due to key handling rather than RC4 itself

RC4 Security



Probability of recovery of plaintext from 2^{24} ciphertexts vs. byte position

RC4 Weaknesses

- Not tamper resistant.
 - Solution: use a MAC.
- XOR of ciphertexts with same key yields XOR of plaintexts.
 - Solution: hash key with nonce.
- Fluhrer, Mantin and Shamir Attack
 - Initial keystream is non-random.
 - If key is simply concatenated with nonce, then key can be recovered.
 - Used to break WEP encryption used by 802.11b wireless networks.

Pseudorandom Number Generation

- **Principles of Pseudorandom Number Generation**

- The Use of Random Numbers
- TRNGs, PRNGs, and PRFs
- PRNG Requirements
- Algorithm Design

- **Pseudorandom Number Generators**

- Linear Congruential Generators
- Blum Blum Shub Generator

- **Pseudorandom Number Generation Using a Block Cipher**

- PRNG Using Block Cipher Modes of Operation
- ANSI X9.17 PRNG

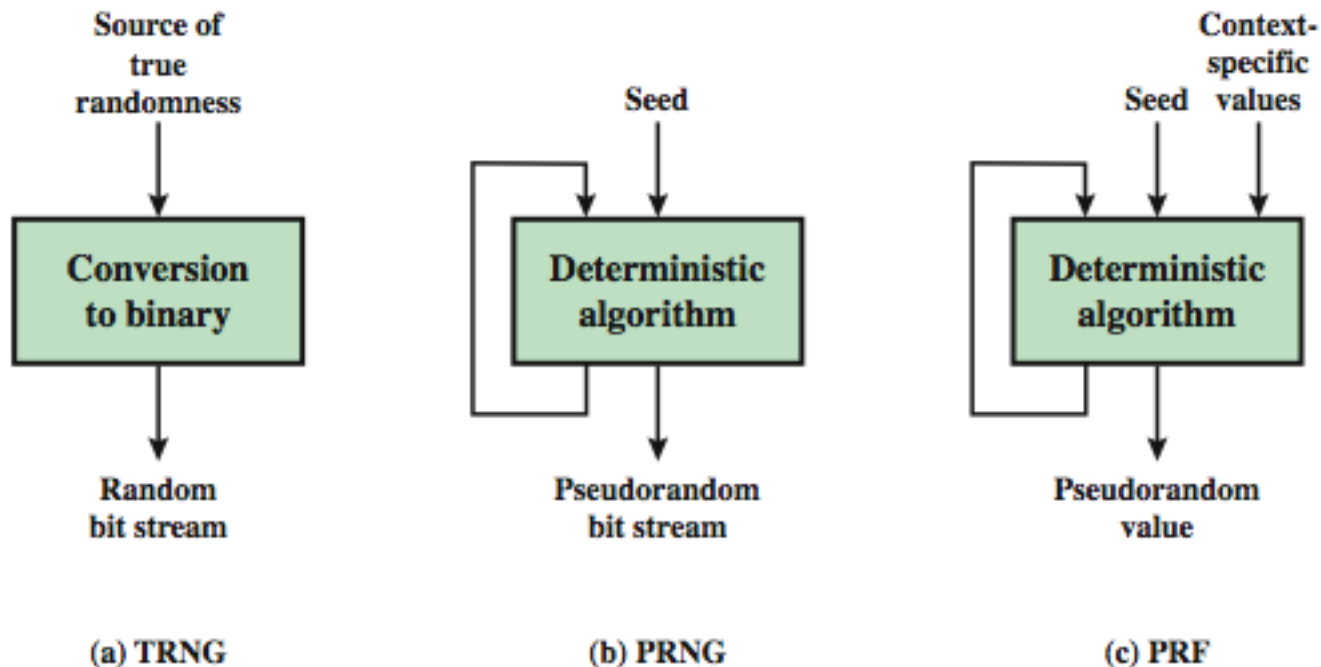
Random Numbers

- many uses of random numbers in cryptography
 - nonces in authentication protocols to prevent replay
 - session keys
 - public key generation
 - keystream for a one-time pad
- in all cases its critical that these values be
 - statistically random, uniform distribution, independent
 - unpredictability of future values from previous values
- true random numbers provide this
- care needed with generated random numbers

Pseudorandom Number Generators (PRNGs)

- often use deterministic algorithmic techniques to create “random numbers”
 - although are not truly random
 - can pass many tests of “randomness”
- known as “pseudorandom numbers”
- created by “Pseudorandom Number Generators (PRNGs)”

Random & Pseudorandom Number Generators



PRNG Requirements

- randomness
 - uniformity, scalability, consistency
- unpredictability
 - forward & backward unpredictability
 - use same tests to check
- characteristics of the seed
 - secure
 - if known adversary can determine output
 - so must be random or pseudorandom number

PRNG Requirements

- Randomness
 - Uniformity – at any point in the generation of the PRN sequence, the occurrence of a zero or a one is equally likely (i.e., $p = 0.5$)
 - Scalability – any test for randomness applicable to a sequence can also be applied to any random subsequence (it should pass)
 - Consistency – the characteristics of the PRN sequence of the PRNG must not depend on the seed used

Linear Congruential Generator

- common iterative technique using:
 - $X_{n+1} = (aX_n + c) \bmod m$
- given suitable values of parameters can produce a long random-like sequence
- suitable criteria to have are:
 - Function should generate a full-period
 - generated sequence should appear random
 - efficient implementation with 32-bit arithmetic
- note that an attacker can reconstruct sequence given a small number of values
- $a = 7, c = 0, m = 32$ sequence = {7, 17, 23, 1, 7, ...}
- $a = 5, c = 0, m = 32$ sequence = {55, 25, 29, 17, 21, 9, 13, 1, 5, ..}

Blum Blum Shub Generator

- based on public key algorithms
- use least significant bit from iterative equation:
$$X_0 = s^2 \bmod n$$
$$\text{for } i = 1 \text{ to } \infty$$
$$x_i = x_{i-1}^2 \bmod n$$
$$B_i = X_i \bmod 2$$
 - where $n = p \cdot q$, and primes $p \equiv q \equiv 3 \bmod 4$
- unpredictable, passes **next-bit** test
- security rests on difficulty of factoring N
- is unpredictable given any run of bits
- slow, since very large numbers must be used
- too slow for cipher use, good for key generation

Blum Blum Shub Generator

- $n = 192649 = 383 * 503$, $s = 101355$

i	X_i	B_i
0	20749	
1	143135	1
2	177671	1
3	97048	0
4	89992	0
5	174051	1
6	80649	1
7	45663	1
8	69442	0
9	186894	0
10	177046	0

i	X_i	B_i
11	137922	0
12	123175	1
13	8630	0
14	114386	0
15	14863	1
16	133015	1
17	106065	1
18	45870	0
19	137171	1
20	48060	0

Using Block Ciphers as PRNGs

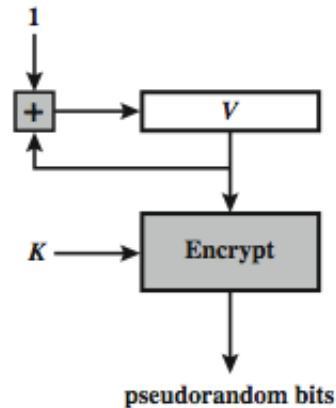
- for cryptographic applications, can use a block cipher to generate random numbers
- often for creating session keys from master key

- CTR

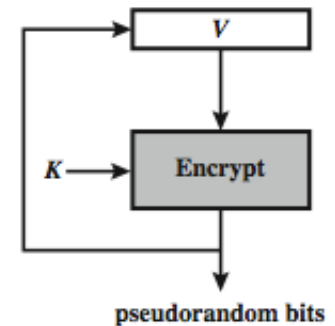
- $X_i = E_K[V_i]$

- OFB

- $X_i = E_K[X_{i-1}]$

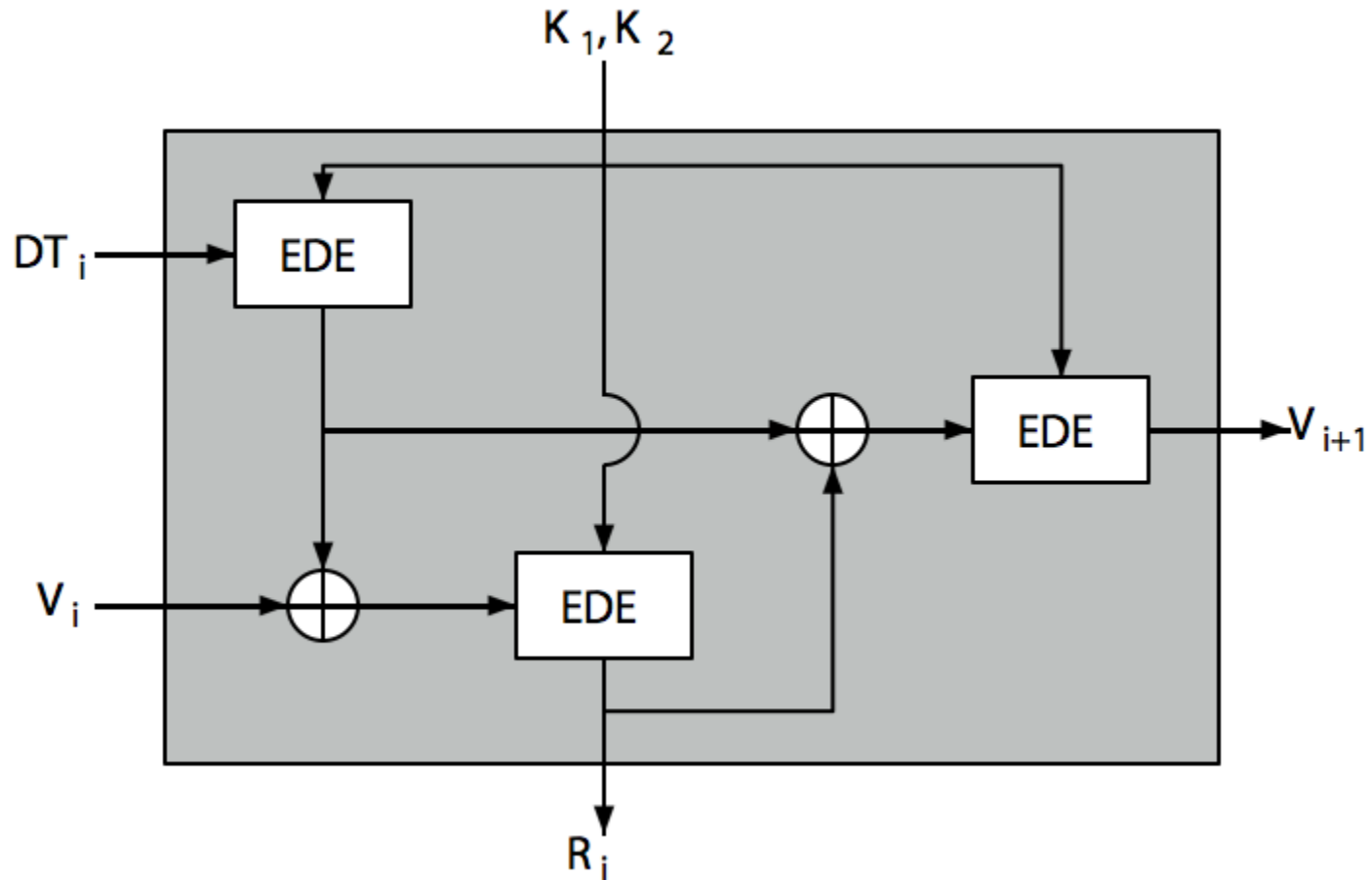


(a) CTR Mode



(b) OFB Mode

ANSI X9.17 PRG



Dti = date and time

Reference

- Cryptography and Network Security-Principles and Practice, 8th Edition, by Stallings William, published by Pearson, 2020
- Cryptography and Network Security, 3rd Edition, by Behrouz A Forouzan and Depdeep Mukhopadhyay, published by McGrawHill, 2015
- Cryptography and Network Security, 3rd edition, by Atul Kahate, published by McGrawHill, 2017